Aluno: Matheus Melle Tártari

- fs/read_write.c
  - SYSCALL_DEFINE3(read, …) →

```
SYSCALL_DEFINE3(read, unsigned int, fd, char __user *, buf, size_t, count)
```

  - ksys_read() →

```
ssize_t ksys_read(unsigned int fd, char __user *buf, size_t count)
```

  - vfs_read() →

```
ssize_t vfs_read(
        struct file *file, char __user *buf, size_t count, loff_t *pos
)
```

  - __vfs_read() →

  - file→f_op→read() →

```
ret = file->f_op->read(file, buf, count, pos);
```

- drivers/tty/tty_io.c
  - tty_read() →

```
static ssize_t tty_read(struct kiocb *iocb, struct iov_iter *to)
```

  - iterate_tty_read() →

```
static ssize_t iterate_tty_read(
        struct tty_ldisc *ld,
        struct tty_struct *tty,
        struct file *file,
        struct iov_iter *to
)
```

  - ld→ops→read →

```
size = ld->ops->read(tty, file, kernel_buf, size, &cookie, offset);
```

- drivers/tty/n_tty.c
  - n_tty_read() →

```
static ssize_t n_tty_read(
        struct tty_struct *tty,
        struct file *file,
        u8 *kbuf,
        size_t nr,
        void **cookie,
        unsigned long offset
)
```

– leitura do vetor read_buf[] em read_buf() para a struct n_tty_data →

```
static inline u8 read_buf(struct n_tty_data *ldata, size_t i) {
        return ldata->read_buf[MASK(i)];
}
```

a) Qual função realiza a leitura na N_TTY?
R: A função read_buf()

b) Em que situação o processo precisa esperar?
R: Quando não há dados para serem lidos no buffer da N_TTY. Nesse caso, o processo é fica em estado de espera e aguarda a chegada de novos dados. Função `n_tty_check_unthrottle`

c) O processo continua consumindo CPU enquanto aguarda? Explique.
R: Não. Quando o processo precisa esperar, ele é colocado em estado de espera e o processador pode executar outros processos. Não fica consumindo CPU enquanto aguarda.

d) O que precisa acontecer para que o processo possa voltar a executar?
R: Novos dados precisam chegar ao buffer da N_TTY, fazendo com que o processo seja acordado